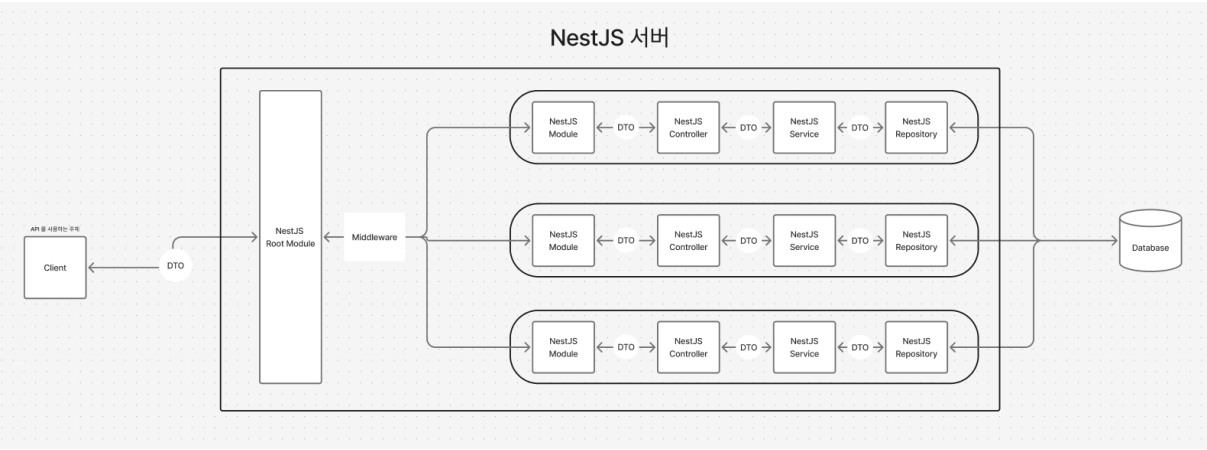
Repository



✅ Repository란?

- **DB 접근 로직을 분리**해서 관리하는 클래스
- Service는 **비즈니스 로직**, Repository는 **DB 쿼리 로직** 전담

> 📌 Controller → Service → Repository → DB

💡 왜 Repository를 쓸까?

- DB 접근 로직이 Service 안에 **섞이면 코드가 복잡**
- 분리하면 **Service는 로직 집중**, Repository는 **쿼리 전담**
- **ORM/DB 변경 시 영향 최소화**

> ❗ 필수는 아님 → 단순한 로직은 Repository 없이도 OK

📦 예시

```
// user.repository.ts
async findByEmail(email: string): Promise<User | null> {
  return this.dataSource.query(
    'SELECT * FROM users WHERE email = $1 LIMIT 1',
    [email],
```

```
  )[0] || null;
}
```

```
// user.service.ts
const user = await this.userRepository.findByEmail(email);
```

> 내부 구현이 SQL이든 Prisma든 Service는 몰라도 됨

✅ Repository의 장점

1. DB 로직 분리 → **유지보수 용이**
2. 쿼리 복잡도 증가 시 → **Service 코드 깔끔하게 유지**
3. **ORM 변경 시 대응 쉬움**
4. **의도 중심 메서드명**으로 코드 가독성 향상

⚠️ Repository의 단점

1. **단순한 CRUD에선 불필요한 레이어**
2. ORM 기능을 **그대로 감싸면 중복 코드** 발생

🎯 언제 쓰는 게 좋을까?

- 복잡한 쿼리가 많은 도메인
- 여러 서비스가 같은 쿼리 재사용
- 단위 테스트 작성 시
- ORM 변경 가능성이 실제로 존재할 때

📌 정리

- Repository는 **전략적으로 사용하는 패턴**
- 복잡도에 따라 도입 여부를 결정
- NestJS에서도 **상황에 따라 유연하게 활용**하면 좋음